

Persistencia en .NET

Entity Framework Core + Dapper

Material de trabajo:

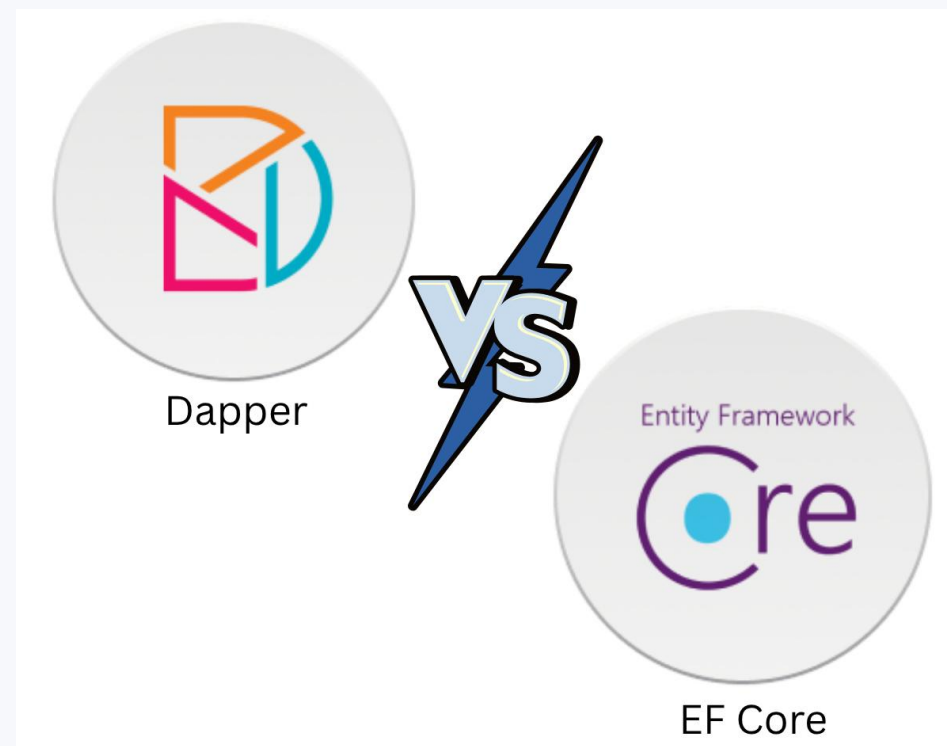
- ASP.NET Core Web API · .NET 10
- SQL Server
- Entity Framework Core
- Dapper
- Caso práctico BiblioNET API

.NET 10

EF Core

Dapper

SQL Server



De ADO.NET a tecnologías de persistencia modernas.

Objetivo y material de la sesión

02

Qué queremos reforzar y con qué herramientas trabajaremos.

Objetivo

Comprender cómo Entity Framework Core y Dapper simplifican el acceso a datos en aplicaciones .NET y qué nivel de abstracción ofrece cada alternativa.

Durante la sesión veremos

- ORM y micro-ORM
- Entity Framework Core
- DbContext, entidades y LINQ
- Code First, Migrations y DB First
- Dapper y Stored Procedures
- CRUD, transacciones y benchmark

Material

ASP.NET Core Web API

exponer endpoints de prueba

SQL Server

base BiblioNETS2DB

EF Core

modelo, LINQ y migrations

Dapper

SP y mapping automático

Visual Studio

depurar el flujo

De ADO.NET al mapping automático

Recordatorio breve de la sesión anterior.

03

¿Qué hacíamos con ADO.NET?

- SqlConnection
- SqlCommand
- SqlParameter
- SqlDataReader
- Mapping hacia objetos

```
while (await reader.ReadAsync())
{
    libros.Add(new Libro
    {
        LibroId = reader.GetInt32(...),
        Titulo = reader.GetString(...),
        Stock = reader.GetInt32(...)
    });
}
```

¿Tenemos que realizar siempre este trabajo manualmente?

¿Qué es un ORM?

04

Una respuesta al problema de traducir entre objetos y tablas.



ORM · Object-Relational Mapper

Un ORM permite mapear datos relacionales hacia objetos de la aplicación y abstraer parte del código necesario para consultar y persistir información.

Entity Framework Core

ORM completo

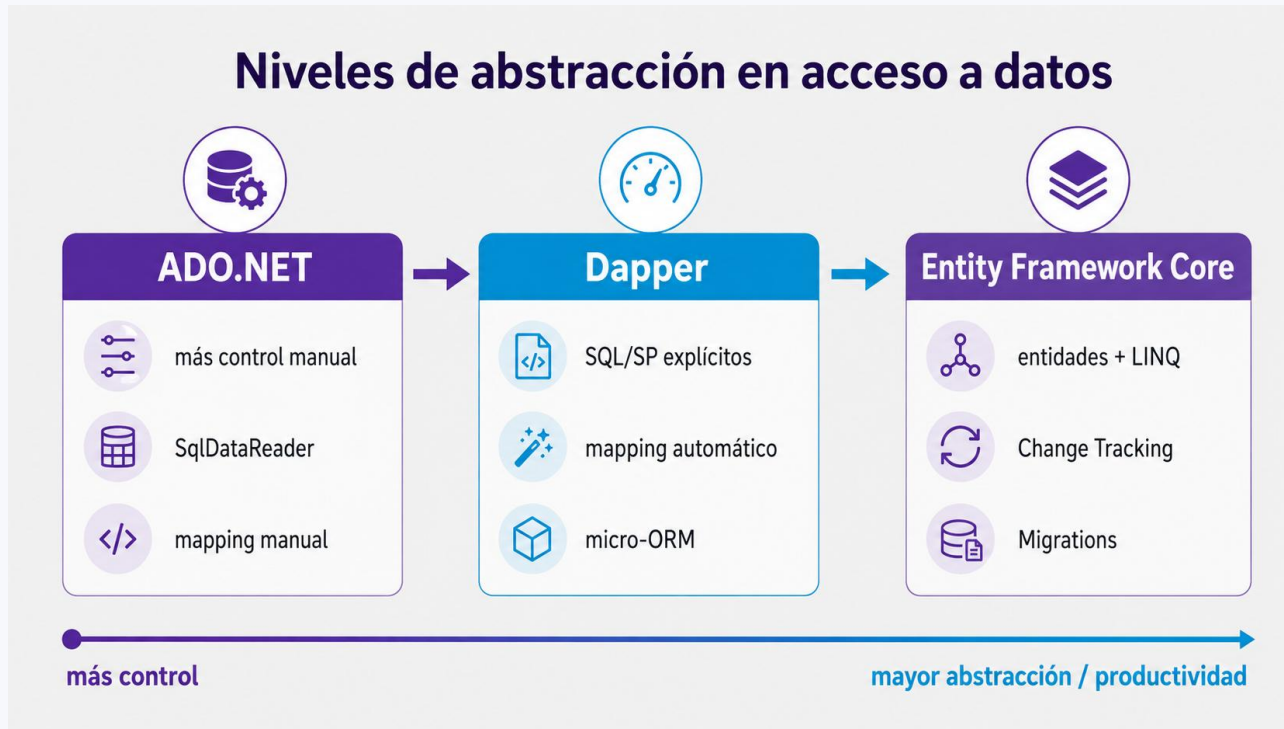
Dapper

Micro-ORM

Diferentes niveles de abstracción

05

Más abstracción significa delegar más detalles técnicos a la herramienta.



Abstracción

En programación, abstraer significa ocultar detalles de implementación para trabajar con conceptos de más alto nivel.

Productividad

Capacidad de resolver más rápido tareas comunes, reduciendo código repetitivo y errores accidentales.

Más abstracción no significa mejor o peor; cambia el balance entre control, productividad y responsabilidad técnica.

Entity Framework Core

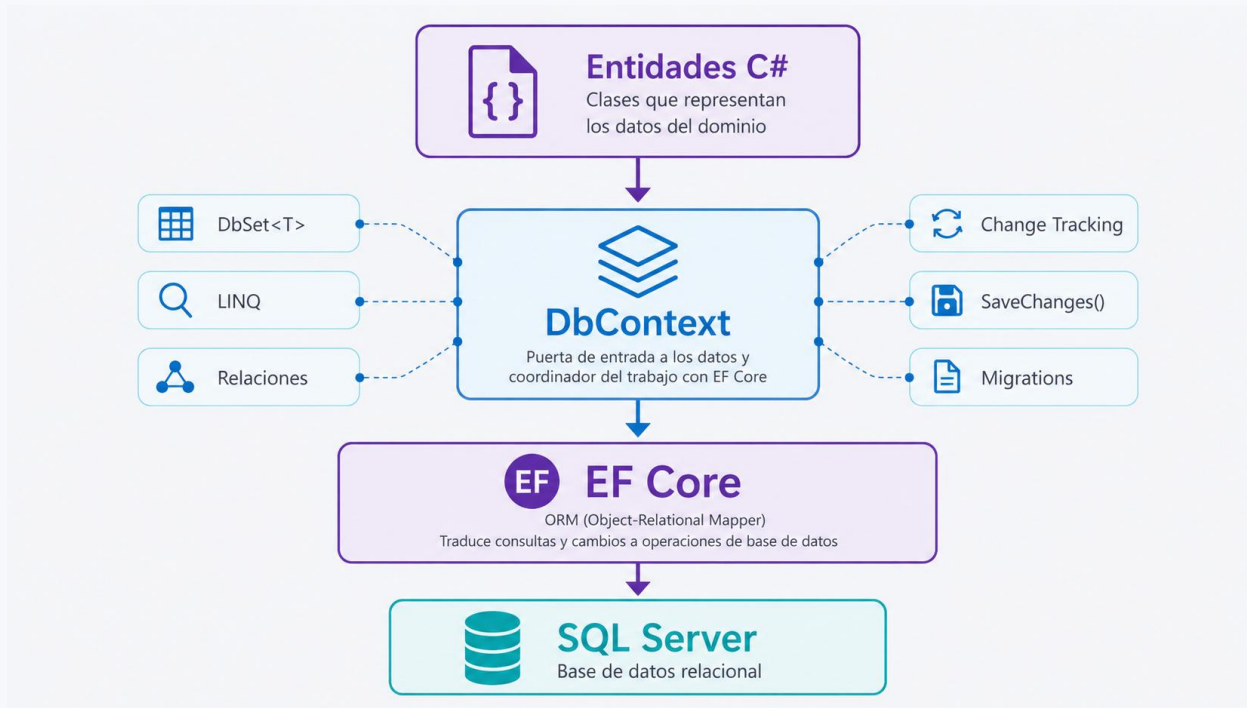
ORM de Microsoft para trabajar con datos desde .NET.

06

Conceptos principales

- Entidades
- DbContext
- DbSet
- LINQ
- Relaciones
- Change Tracking
- SaveChanges
- Migrations

Con EF Core normalmente expresamos qué datos necesitamos con C# y LINQ, y EF traduce esas operaciones hacia SQL Server.



Entidades, DbContext y LINQ

07

Tres piezas clave para entender el flujo de EF Core.

```
public sealed class Libro
{
    public int LibroId { get; set; }
    public int GeneroId { get; set; }
    public string Titulo { get; set; }
    public int Stock { get; set; }
    public Genero Genero { get; set; }
}
```

Entidad C#

```
public sealed class BiblioNetDbContext
    : DbContext
{
    public DbSet<Libro> Libros
        => Set<Libro>();
}
```

DbContext / DbSet

```
var libros = await db.Libros
    .AsNoTracking()
    .Where(x => x.Activo)
    .OrderBy(x => x.Titulo)
    .Select(x => new LibroDto(...))
    .ToListAsync();
```

LINQ → SQL

Code First + Migrations

08

Versionar el esquema de base de datos desde el modelo C#.

Comandos utilizados

```
dotnet ef migrations add InitialCreate
dotnet ef database update
```

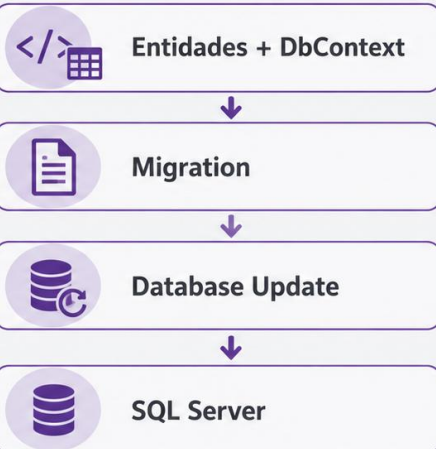
Ventajas frente a cambios manuales

- Registra la evolución del esquema
- Permite versionar cambios junto al código
- Reduce diferencias entre ambientes
- Crea tablas, relaciones, índices y restricciones
- Mantiene historial en __EFMigrationsHistory

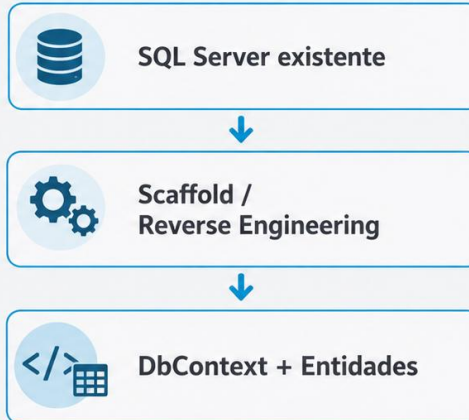
```
public partial class InitialCreate : Migration
{
    /// <inheritdoc />
    protected override void Up(MigrationBuilder migrationBuilder)
    {
    }
    /// <inheritdoc />
    protected override void Down(MigrationBuilder migrationBuilder)
    {
    }
}
```

Code First vs DB First en EF Core

CODE FIRST



DB FIRST / SCAFFOLD

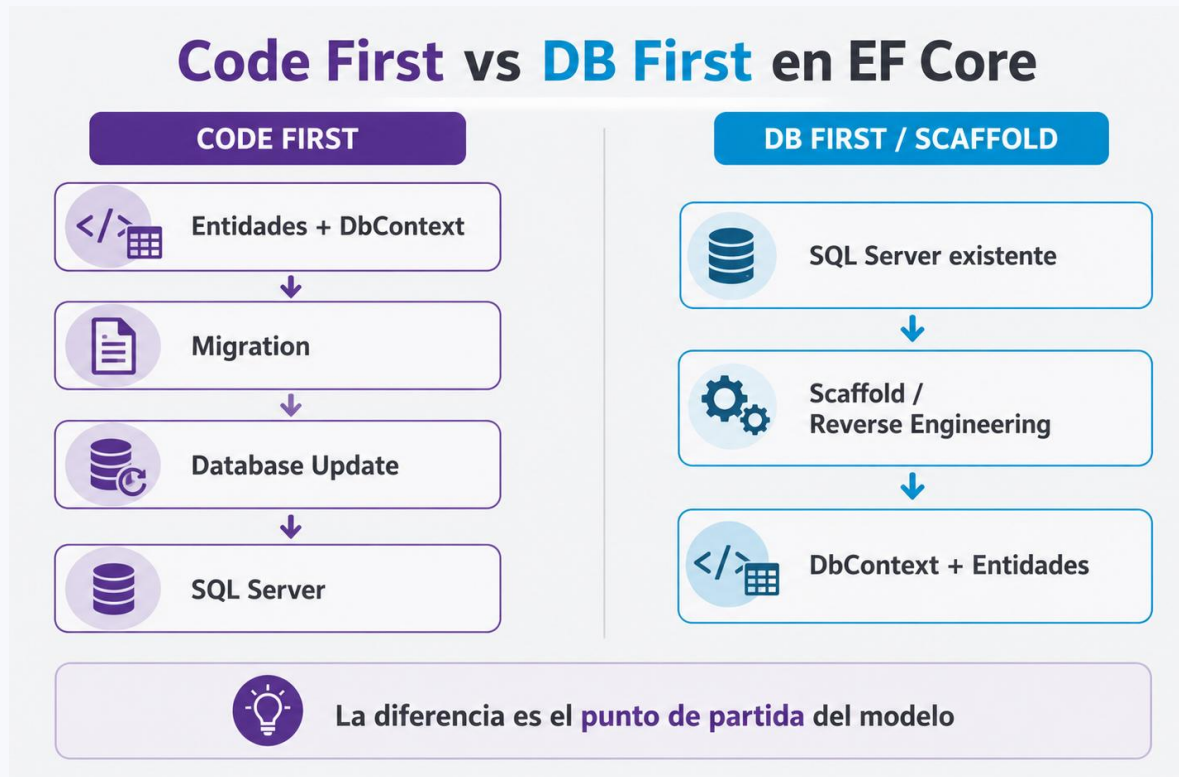


La diferencia es el **punto de partida** del modelo

DB First + Scaffold

09

Cuando la base de datos ya existe y es el punto de partida.



Reverse Engineering

EF Core puede generar un DbContext y entidades a partir de un esquema existente.

Ventajas

- Útil en bases existentes
- Acelera el inicio del modelo
- Reconoce tablas y relaciones

Consideraciones

- El diseño de BD manda
- Cuidado con sobrescrituras
- El modelo puede requerir limpieza manual



Dapper

10

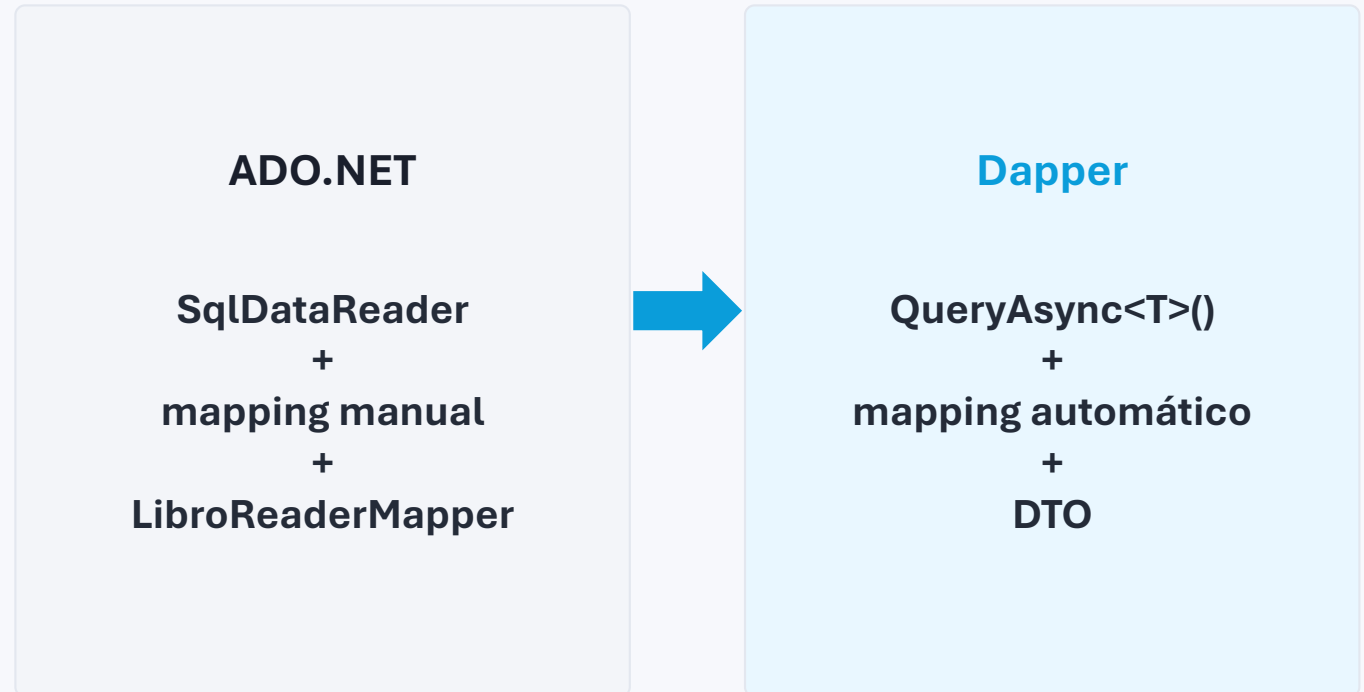
Micro-ORM ligero construido sobre conexiones ADO.NET.

Conserva

- SqlConnection
- SQL explícito
- Stored Procedures
- Parámetros

Simplifica

- Ejecución de comandos
- Parametrización
- Mapping hacia objetos
- Menos código repetitivo



Dapper + Stored Procedures

11

SQL explícito y mapping automático sin escribir un SqlDataReader por cada consulta.

```
var result = await connection.QueryAsync<LibroDto>(
    "dbo.usp_Libro_Listar",
    CommandType.StoredProcedure);
```

```
return await connection.QuerySingleAsync<int>(
    "dbo.usp_Libro_Insertar",
    parametros,
    CommandType.StoredProcedure);
```

Operaciones frecuentes

QueryAsync<T>

múltiples registros

QuerySingleAsync<T>

un único resultado

ExecuteAsync

comandos sin entidades

Dapper puede trabajar directamente con Stored Procedures y parámetros.

EF Core vs Dapper

Misma necesidad, distinta filosofía.

```
db.Libros
  .AsNoTracking()
  .Where(x => x.Activo)
  .Select(x => new LibroDto(...))
  .ToListAsync();
```

```
connection.QueryAsync<LibroDto>(
  "dbo.usp_Libro_Listar",
  CommandType.StoredProcedure);
```

Entity Framework Core	Dapper
Modelo y entidades	SQL / SP explícitos
LINQ	SQL controlado
Change Tracking	Sin tracking integrado
Relaciones integradas	Mapping liviano
Migrations	No incluye migrations
Mayor abstracción	Menor abstracción

CRUD con EF Core y Dapper

13

Ambos persisten datos; cambia cómo expresamos la operación.

Entity Framework Core

```
var libro = new Libro { ... };  
db.Libros.Add(libro);  
await db.SaveChangesAsync();
```

EF administra el estado de la entidad mediante el DbContext.

Dapper

```
var id = await connection  
    .QuerySingleAsync<int>("dbo.usp_Libro_Insertar",  
        parametros,  
        commandType: StoredProcedure);
```

Dapper ejecuta directamente la operación definida.

El resultado puede ser el mismo; la diferencia está en la forma de expresar y administrar el acceso a datos.

Transacciones

14

Una operación de negocio puede involucrar múltiples cambios.

Caso de la demo

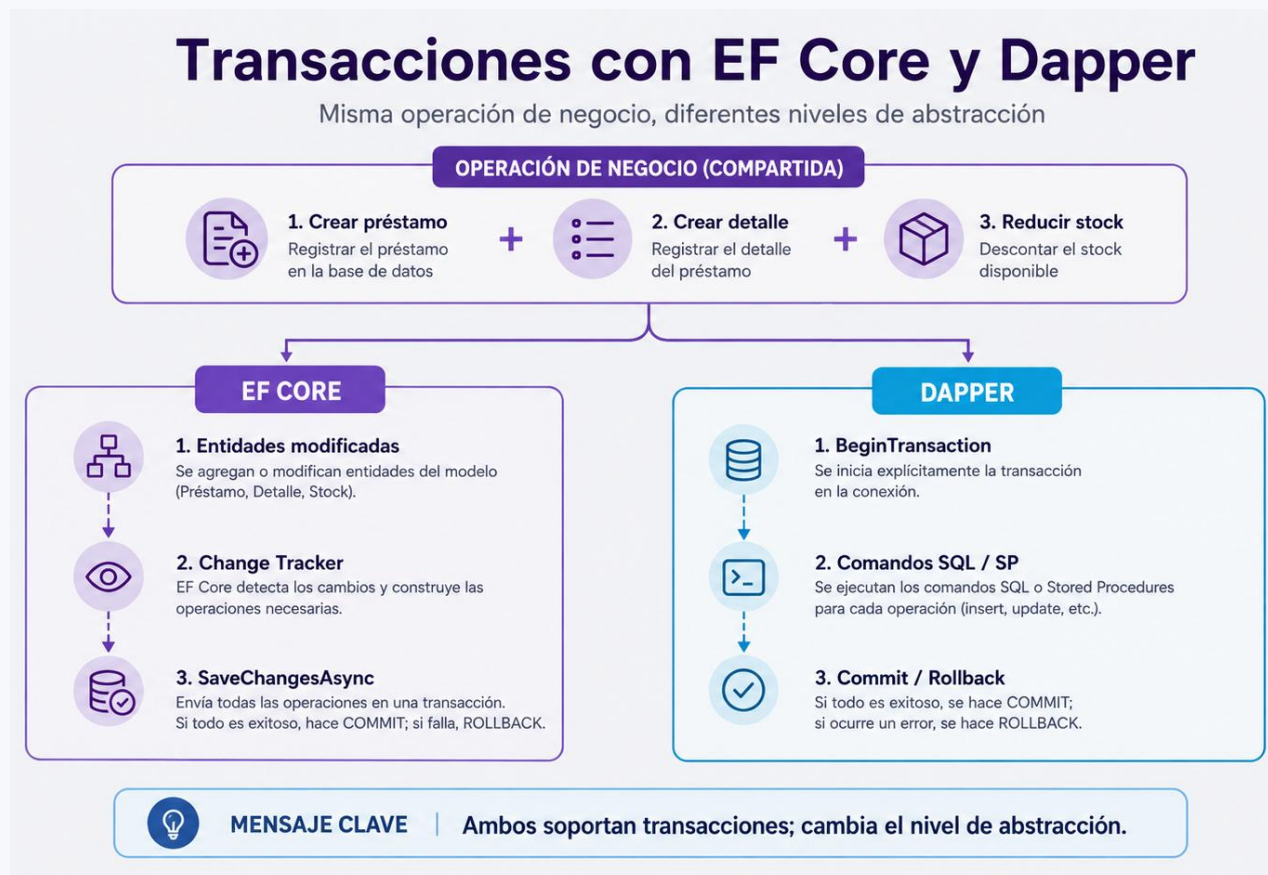
- Crear préstamo
- Crear detalle
- Reducir stock del libro

Change Tracking

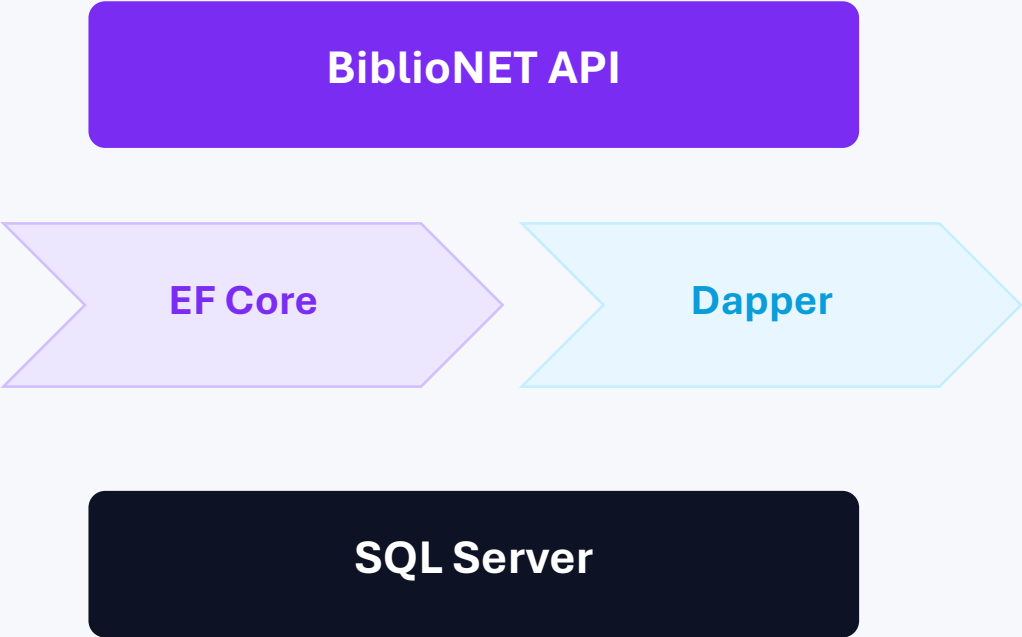
EF Core registra qué entidades se agregaron, modificaron o eliminaron dentro del DbContext para persistirlas en `SaveChangesAsync()`.

Idea clave

EF Core y Dapper pueden trabajar con transacciones; cambia el nivel de abstracción y el control que asumimos.



Caso práctico de la sesión.



CRUD de libros

Stored Procedures

Relaciones

Transacciones

Migrations

Benchmark de lectura

Benchmark: diseño de BD antes que framework

Mediciones locales sobre 1 000 000 de registros idénticos.

Escenario

Dos tablas con los mismos datos. Una sin índice secundario específico y otra con un índice diseñado para la consulta.

COUNT · Mediana

Tecnología	Sin índice	Con índice
EF Core	29.833 ms	3.083 ms
Dapper	29.093 ms	1.695 ms

TOP 100 · Mediana

Tecnología	Sin índice	Con índice
EF Core	3.047 ms	1.812 ms
Dapper	1.491 ms	0.562 ms



Demo en vivo

17

La presentación explica el patrón; la ejecución muestra el comportamiento real.

1

Entity Framework Core

Consulta LINQ, CRUD y SaveChangesAsync

2

Dapper

Stored Procedures, mapping automático y CRUD

3

Transacciones

EF Core y Dapper sobre la misma operación

4

Benchmark

Sin índice vs con índice

5

Migrations

InitialCreate y database update

BiblioNET API → EF / Dapper → SQL Server

Mitos y criterios para elegir

18

Cierre: no buscamos un ganador universal, sino criterio técnico.

Mitos frecuentes

“Dapper siempre es más rápido.”

“EF Core es lento.”

“Dapper no maneja transacciones.”

“Dapper es solo para SQL inline.”

“Agregar un índice siempre mejora todo.”

Cuándo considerar EF Core

- CRUD convencional
- Modelos con relaciones
- Alta productividad
- LINQ
- Change Tracking
- Migrations

Cuándo considerar Dapper

- SQL / SP existentes
- Consultas controladas
- Reporterías
- Menor abstracción
- Modernización gradual

No buscamos elegir una tecnología ganadora, sino la herramienta adecuada para cada escenario.